This will make the `test` task invoke the `build` task before it runs, but it doesn't check to see if the `build` task has already been invoked. Suppose in your Cakefile you have three tasks: `deploy`, `build`, and `test`:

```
task 'build', ->
  console.log 'built'

task 'test', ->
  invoke 'build'

task 'deploy', ->
  invoke 'build'
  invoke 'test'
```

The `deploy` task should do a deployment only if the `build` and `test` tasks both complete successfully, and the `test` task requires a completed build to run, so that should run only if the `build` task is completed. Unfortunately, if you use the tasks as defined previously, then the `build` task will be invoked twice when you deploy:

```
> cake deploy
# built
# built
```

`invoke` does *only* what it says in the name: invokes the other task. It doesn't care if the other task has been run once, twice, or 100 times. Because Cake doesn't provide any way to tell a task to run only once each time, you have to do it yourself, with a variable. This isn't pretty, but it works:

```
built = false

task 'build', ->
  return if built
  built = true
```

If built is true, then the task has already been run.

Now you're automating your build with Cake. Depending on how complicated your build is, you might need to dive into some of the more advanced Node.js libraries for doing things like spawning, forking, and executing external processes. The official Node.js documentation is the best place to get up-to-date information on those.